

Smart Pantry- IoT Inventory Tracker for Kitchen Essentials

Kashaf Qureshi
Alma Mater Studiorm-
Universita di Bologna
Bologna, Italy
kashaf.qureshi2@studio.unibo.it

Wajeeha Kazmi
Alma Mater Studiorm-
Universita di Bologna
Bologna, Italy
wajeeha.kazmi@studio.unibo.it

Abstract— *A compact and low-cost IoT system designed to track food stock levels (like flour, rice, coffee, etc.) by detecting weight variations and container usage. The device sits under pantry containers and is able to detect when and how much of an item is used. Additionally, the project demonstrates the concurrent use of multiple data communication protocols—HTTP and MQTT—highlighting their interoperability and practical integration within the same system. The system's capabilities are further extended through the adoption of W3C Web of Things (WoT) protocols, enhancing interoperability, and a dedicated Telegram bot interface that facilitates direct user engagement for real-time status monitoring and alert management.*

Keywords— *Internet of Things (IoT), ESP32, Sensors, Python, HTTP/MQTT Protocols, Forecasting, Web of Things, Telegram Bot*

I. INTRODUCTION

Smart pantry is an important concept to efficiently control inventory in the kitchen. People don't have time in their busy schedules to go to the store to buy groceries, and they may forget that a particular item has run out. The decision might not only be to purchase a specific item, but also how much of that item to purchase. This is achieved by examining things inside of the containers. Once they are emptied or hit minimum stocks they need to be re-purchased from the shop.

This project presents an automated smart pantry system built with an ESP32 microcontroller. The device monitors food levels using weight and light sensors, alerting users when supplies run low. Data is transmitted via HTTP and MQTT to a central proxy, which stores the data into InfluxDB and performs future forecasts. Thresholds are adjustable remotely through MQTT, and Grafana will allow us to see stock and consumption trends.

The intent is to illustrate how typical IoT protocols (HTTP, MQTT) and time-series analytics and modern interaction standards can combine to produce a practical, inexpensive system for managing household inventory.

II. PROJECT ARCHITECTURE

The system consists of ESP32-based nodes connected over Wi-Fi, a light and weight sensor. The node continuously monitors the food stock levels with the help of weight sensor HX711 and detects the container access with the help of light sensor.

Weight data is transmitted to a central Python proxy using HTTP, while asynchronous events (e.g., low-stock alerts) are published via MQTT. The proxy server processes this data, logging raw measurements and events into an InfluxDB time-series database. Thresholds for stock alerts are configurable at runtime through MQTT topics, allowing dynamic adjustments without hardware reprogramming.

Logged data is visualized in a Grafana dashboard, displaying real-time inventory levels, usage trends, and alert histories. In parallel, a Python forecasting module analyzes historical consumption patterns using machine learning (e.g., linear regression), predicting depletion dates. These forecasts are stored back in InfluxDB, enabling comparative analysis between predicted and actual usage through Grafana.

This design demonstrates a complete IoT pipeline—from edge sensing (ESP32) to cloud analytics (Python/Forecasting), addressing practical challenges in household inventory management while leveraging standard protocols (HTTP, MQTT) and time-series databases (InfluxDB).

To demonstrate advanced IoT integration, the system's architecture supports two supplementary modules: A W3C Web of Things (WoT) compliant interface that standardizes device interaction, and a Telegram bot that serves as a conversational user interface for remote monitoring and control. These components operate concurrently with the core data pipeline, showcasing a multi-protocol, user-centric IoT ecosystem.



Fig-I: Grafana output of Weight and Light sensor reading

III. PROJECT IMPLEMENTATION

The system consists of an ESP32 microcontroller programmed using the Arduino framework (Platform IO), integrating weight and light sensors with dual-protocol communication (HTTP/MQTT). Runtime configurations (thresholds, sampling rates) are pushed via MQTT, while sensor data is transmitted via HTTP to the Flask proxy. Logging is performed directly to an InfluxDB instance, and forecasting is handled by an external Python script.

A. Hardware Components and Sensor Integration

- Weight Sensing

The system uses an HX711 load cell connected to GPIO18 (DOUT) and GPIO19 (SCK) for precise weight measurements. The HX711.h library^[1] handles 24-bit ADC readings, with calibration managed via `set_scale()` and zeroing using `tare()`. This ensures accurate tracking of item quantities, even for small changes.

- Light Detection

A Light Dependent Resistor (LDR) on GPIO34 (ADC1) detects when the pantry container is opened or closed. The built-in `analogRead()` function measures light intensity, with a configurable threshold (`LIGHT_THRESHOLD = 500`) to distinguish between open and closed states.

B. Communication Protocols

- HTTP for Data Transmission

The `HTTPClient.h` library^[2] sends periodic sensor data to a Flask proxy server. Each payload includes weight, light levels, and a timestamp, formatted as JSON

- MQTT for Event Alerts

The PubSubClient.h library^[3] enables asynchronous communication for critical events. For example, when stock levels fall below a threshold, the ESP32 publishes alerts to topics like smartpantry/lowstock.

- Time Synchronization

Accurate timestamps are ensured using the built-in time.h library, which syncs with pool.ntp.org via NTP. This is critical for correlating sensor data with real-world events.

C. Backend and Data Processing

- Flask Proxy Server

A Python-based proxy server, built with the library acts as the intermediary between the ESP32 and InfluxDB. It receives HTTP data, processes MQTT alerts, and logs everything to InfluxDB using the influxdb_client library^[4].

- InfluxDB stores:
 - Raw sensor data (weight, light).
 - Event logs (e.g., low-stock alerts).
 - Forecast results (depletion_ts).

- Grafana Visualization

Grafana dashboards display real-time metrics, including:

- Weight trends (with lid ON/OFF states).
- Light sensor spikes (indicating container access).

D. Predictive Analytics

An external Python script uses sklearn.linear_model.LinearRegression to analyze historical data and predict when stock will deplete. The script fetches data from InfluxDB via influxdb_client and writes forecasts back to the database for visualization in Grafana. The system uses pandas^[5] to process time-series weight data from InfluxDB, structuring it for analysis by converting timestamps and sorting values. The scikit-learn library^[6] then applies linear regression to this data, modeling consumption patterns to predict when pantry items will run out.

E. Standards-Based Device Integration with W3C Web of Things

The ESP32 microcontroller was augmented to function as a compliant Web Thing within the W3C WoT framework^[7]. This transformation involved hosting a lightweight web server on the device itself, which dynamically serves a Thing Description (TD)—a JSON document that semantically describes the device's capabilities, data points (termed properties), and the hypermedia controls (forms) for accessing them.

This TD exposes the weight and light metrics as read-only properties, making the device self-descriptive and discoverable. Consequently, the data proxy was refactored to act as a WoT Consumer. Instead of passively awaiting HTTP POST requests, it can actively fetch data by retrieving the TD, parsing the available properties, and issuing HTTP GET requests to the endpoints specified within (e.g., <http://192.168.137.164/properties/weight>). This implementation validates a shift from a proprietary, push-based data collection method to a standardized, pull-based paradigm, significantly improving cross-platform interoperability.

F. Conversational User Interface via Telegram Bot

A complementary user interface was developed using the Telegram messaging platform to provide ubiquitous access to system status and controls. A dedicated bot was created using the python-telegram-bot library^[8], which processes user commands asynchronously.

The bot supports two primary imperative commands: the **/status command** triggers an HTTP GET request to the ESP32's embedded web server, fetching and displaying the current weight and light levels directly within the chat. The **/threshold [value]** command allows users to dynamically redefine the low-stock alert level, which is stored as a global variable within the bot's runtime and directly influences the alerting logic. Furthermore, this logic was integrated directly into the data-fetching routine, enabling the system to push immediate notifications to the user's Telegram client upon detecting a breach of the predefined stock level. This chat-based interface eliminates the need for a dedicated mobile application, leveraging a familiar and pervasive platform for effective human-in-the-loop interaction.

IV. RESULTS

The Smart Pantry system was tested under normal household conditions. Both communication protocols, HTTP (for sensor data) and MQTT (for alerts), were evaluated.

Data Pipeline Performance: The end-to-end latency of the system, measured from sensor readout on the ESP32 to storage in InfluxDB, was analyzed. At the start, there's high latency (~1.75 seconds). After that, latency drops to near-zero and remains consistently low (close to 0s), which is typical for local or low-congestion networks (*Figure-II*). This could imply that the initial connection or setup was slow. Later transmissions are stable and fast. The latency data appears dense and sampled frequently, indicating that the ESP32 is sending data at a high rate.

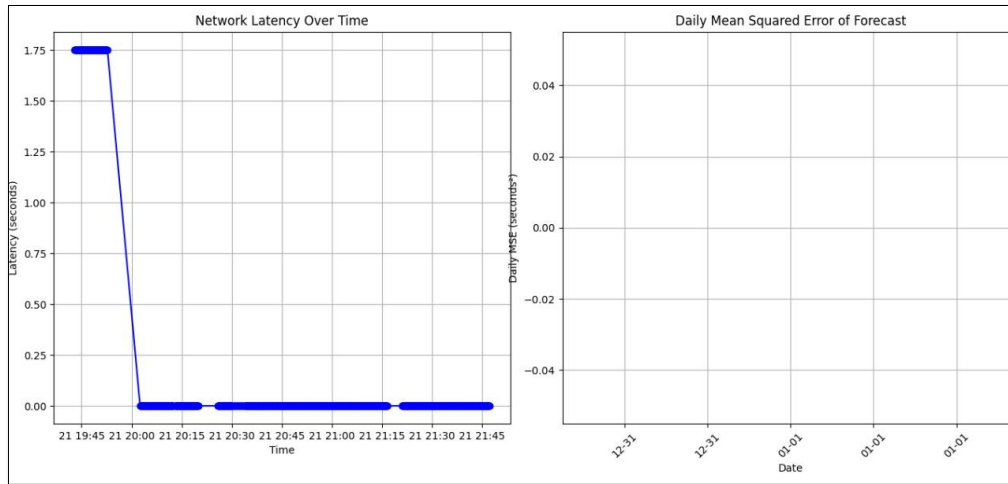


Fig-II Mean Latency and MSE

Bonus Feature Validation: The implementation of the two optional advanced features was successfully validated:

1. **W3C Web of Things (WoT) Integration:** The successful implementation of the W3C Web of Things (WoT) standard was validated through direct testing. The ESP32 device, functioning as a compliant Web Thing, successfully served its Thing Description (TD) and property endpoints. As demonstrated in *Figure-III*, HTTP GET requests to the standardized endpoints `http://192.168.137.164/properties/weight` and `http://192.168.137.164/properties/light` returned live, real-time sensor readings in a consistent JSON format (`{"weight": 1746.42}` and `{"light": 1269}`).

This result confirms that the device is now self-descriptive and interoperable. Any authorized consumer on the network can discover these endpoints via the TD and retrieve sensor data without prior knowledge of the device's internal architecture, fulfilling the core requirement of the W3C WoT standard.

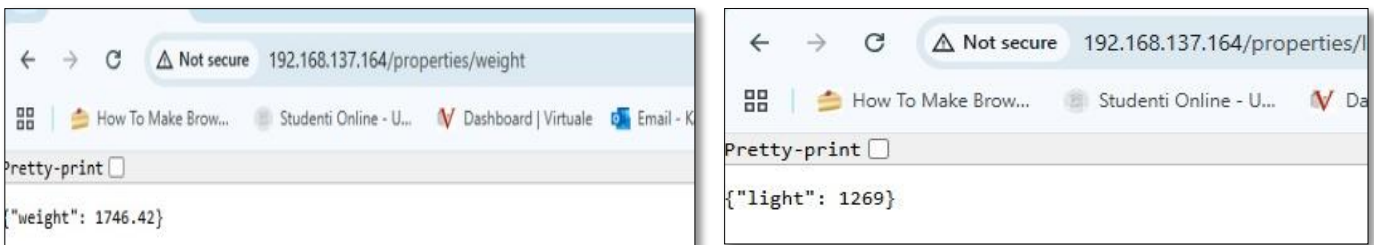


Fig-III: Live responses from the ESP32's endpoints

2. Telegram Bot Interface: The bot's functionality for user interaction and alerting was verified. The `/status` command successfully returned live sensor values (e.g., 268.09 g), while the `/threshold` command dynamically reconfigured the alert level. Most significantly, the system autonomously pushed a **low-stock alert** to the user (Fig-IV), demonstrating a complete monitoring and notification feedback loop.

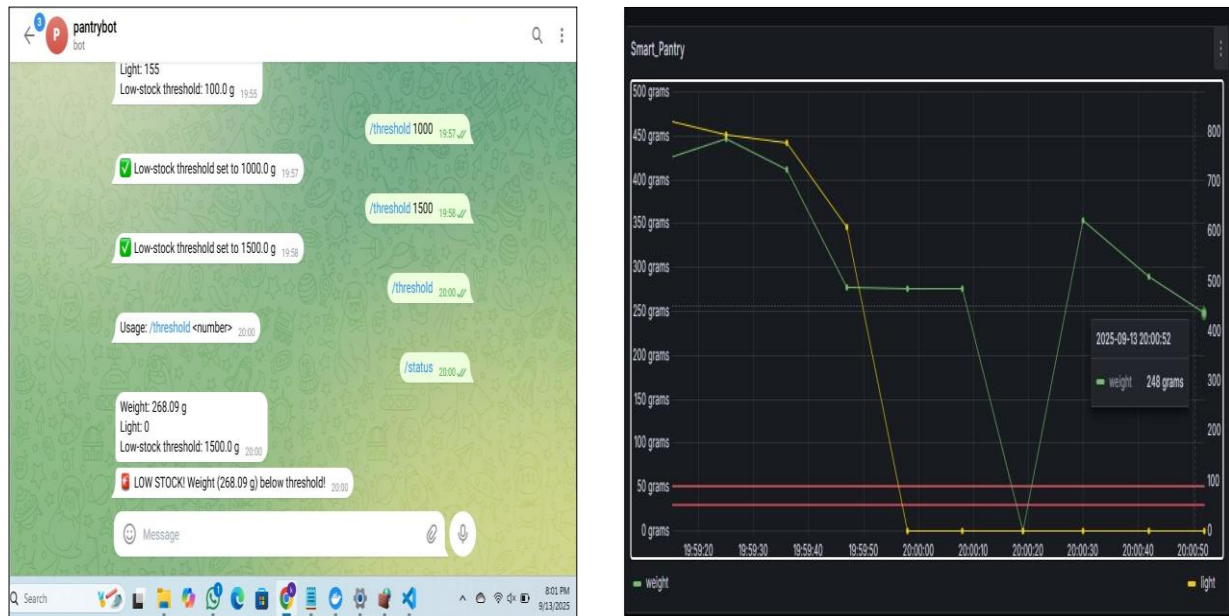


Fig-VI: Telegram Bot Interface Validation

REFERENCES

- [1] HX711 Library <https://github.com/bogde/HX711J>
- [2] HttpClient Library. GitHub <https://github.com/amcewen/HttpClient>
- [3] N. O'Leary, "PubSubClient for Arduino," GitHub, <https://github.com/knolleary/pubsubclient>
- [4] InfluxDB Library <https://github.com/influxdata/influxdb-client-python>
- [5] Pandas Library <https://pandas.pydata.org/>
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. <https://scikit-learn.org/>
- [7] W3C Web of Things (WoT) Architecture. W3C Recommendation. <https://www.w3.org/TR/wot-architecture/>
- [8] Python Telegram Bot library. <https://github.com/python-telegram-bot/python-telegram-bot>
- [9] WebServer library for ESP32 (Arduino Core). <https://github.com/espressif/arduino-esp32/tree/master/libraries/WebServer>
- [10] <https://lucassardois.medium.com/handling-iot-data-with-mqtt-telegraf-influxdb-and-grafana-5a431480217>
- [11] [IoT PAPERS\Time series prediction in IoT a comparative study of federated versus centralized learning.pdf](#)